PetCare 宠物店管理系统 · 第二阶段成果展示（应用侧）

一、系统概览

PetCare 是一个宠物店管理系统。本阶段在数据库设计成果之上，完成了 Web 应用的构建，覆盖以下业务：

- 登录鉴权与角色权限
- 商品管理（查询 / 新增 / 编辑 / 停售）
- 库存管理（查询 / 流水 / 补货入库）
- 订单管理（开单 / 查询 / 详情 / 状态）
- 会员管理（建档 / 改资料 / 积分调整）
- 员工管理与经营报表

系统面向「店员」「店长」两类角色，提供不同的界面与操作权限。

二、系统架构

应用采用四层分层架构，业务层不直接编写 SQL，通过数据访问层（DAO）访问数据库：

```
页面层     app/routes/       接收表单 → 调用业务层 → 渲染页面 / 提示
   ↓
业务层     app/services/     登录鉴权、参数校验、业务规则、事务边界、角色权限
   ↓
数据访问层  dao/              参数化 SQL，事务内执行，不提交 / 回滚 / 关闭连接
   ↓
数据库     SQL Server        受限账号（petcare_auth / petcare_clerk / petcare_manager）
```

权限在**应用层**与**数据库层**各校验一次，构成「双重防线」：应用层由 `roles_required` 装饰器控制，数据库层由受限账号的最小权限控制。

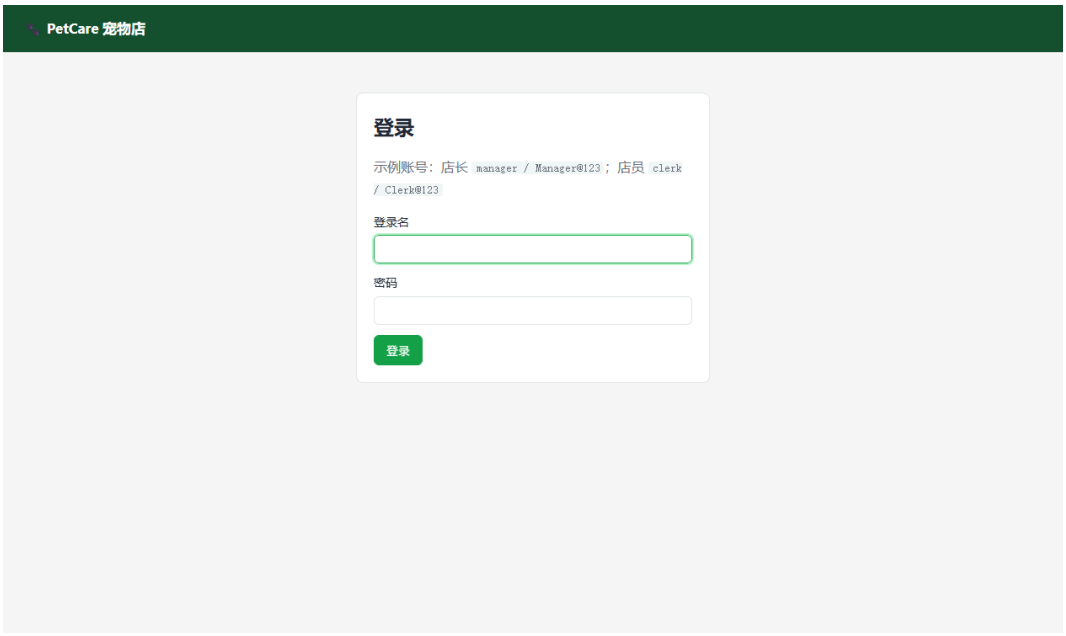
三、核心功能

3.1 登录鉴权

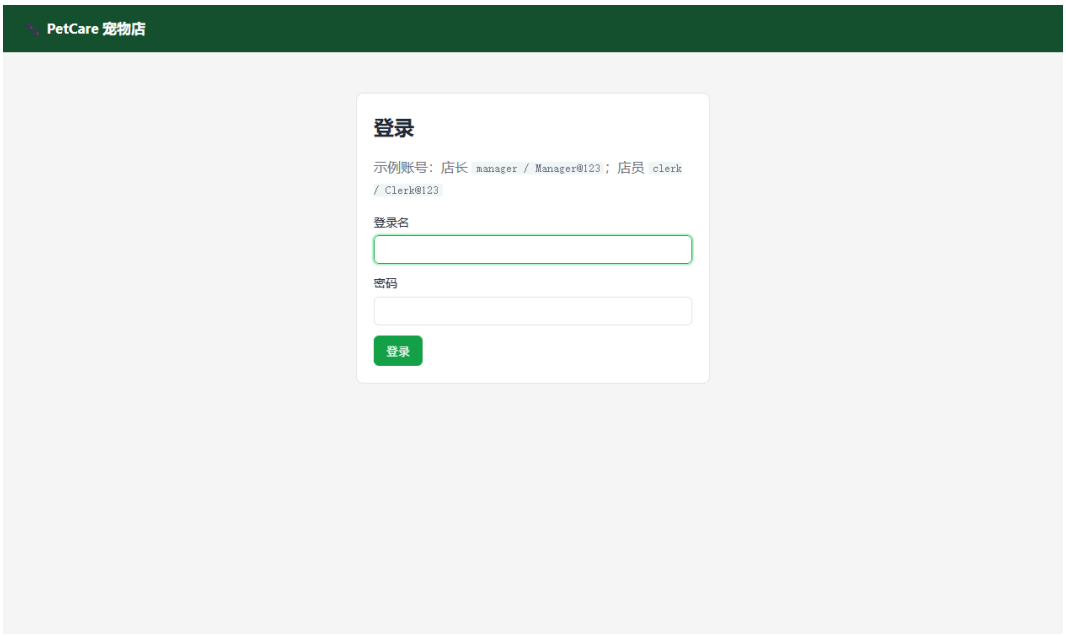
用户通过 `/login` 登录。系统使用独立连接只读凭据视图 `v_employee_login`，以 `werkzeug` 的 `check_password_hash` 校验密码（scrypt 哈希），校验通过后将用户信息写入会话：

```
with get_db().auth_read() as conn:                # 登录连接与业务连接分离
    cred = EmployeeDAO().get_credentials(conn, login_name)
if not check_password_hash(cred["password_hash"], password):
    raise DAOError(ErrorCode.UNAUTHENTICATED, "用户名或密码错误")
session["role"] = cred["role"]                    # 「店长」/「店员」
```

未登录访问受保护页面时，系统重定向到登录页，并携带回跳地址。



登录页



未登录访问被重定向到登录页

3.2 角色与权限

导航栏根据角色条件渲染，店长比店员多出「员工」「报表」两个入口：

```
{% if current_user.role == '店长' %}
  <a href="{{ url_for('employees.list') }}">员工</a>
  <a href="{{ url_for('reports.index') }}">报表</a>
{% endif %}
```

角色	登录账号	可用功能
店员	clerk	商品/库存查看、开单、订单查询、会员办理与积分调整

角色	登录账号	可用功能
店长	`manager`	店员全部功能 + 新增/编辑/停售商品、补货入库、员工管理、经营报表

写操作在路由层以 `@roles_required("店长")` 拦截。



店员工作台（无「员工/报表」入口）



店长工作台（含「员工/报表」入口）

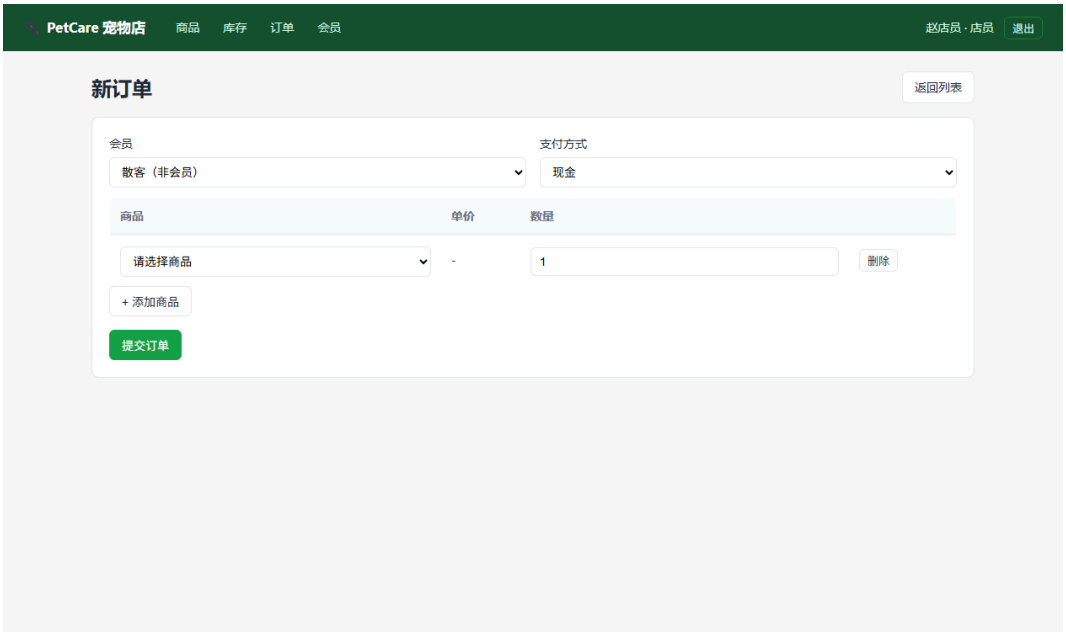
3.3 开单流程

开单是系统的核心业务流程，完整链路如下：

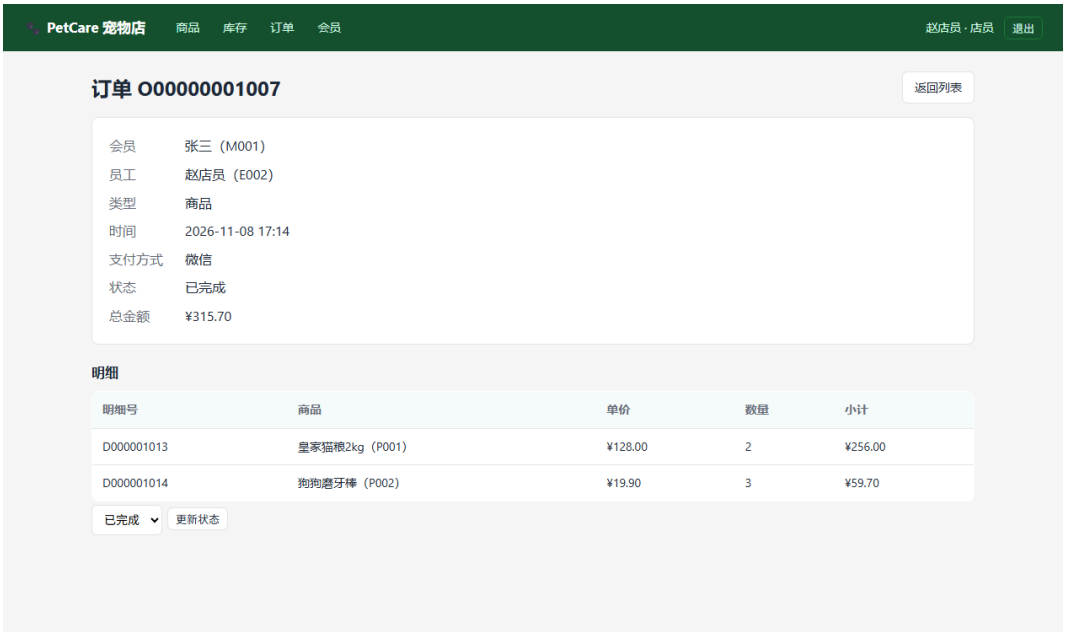
```
/orders/new 表单
  → order_service.create_order()
     → db.transaction(角色)                          # 事务边界在业务层
        → OrderDAO.create_with_details()
           1. 锁定商品与库存（UPDLOCK / ROWLOCK）        # 防并发抢库存
           2. 校验：在售 / 库存记录存在 / 库存充足
           3. 价格取 Product.price，不接受前端传入
           4. 相同商品自动合并数量
           5. 写入 Orders + OrderDetail（subtotal 为计算列）
           6. 扣减 Inventory（带 quantity >= ? 守卫）
           7. 写入 InventoryLog 销售流水
```

两个关键设计：

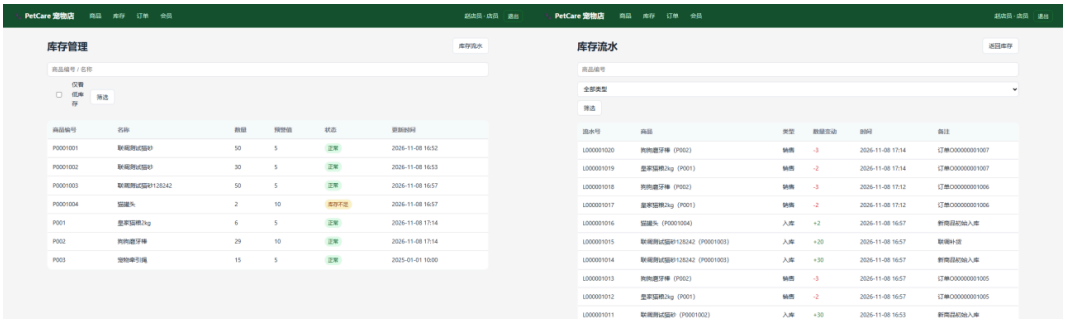
1. **开单人不信前端**：`employee_id` 从会话 `session["employee_id"]` 获取，不从表单传入，防止越权冒名。
2. **整单回滚**：任一环节校验失败（停售 / 库存不足），事务整体回滚，订单、明细、库存、流水均不留脏数据。



开单表单



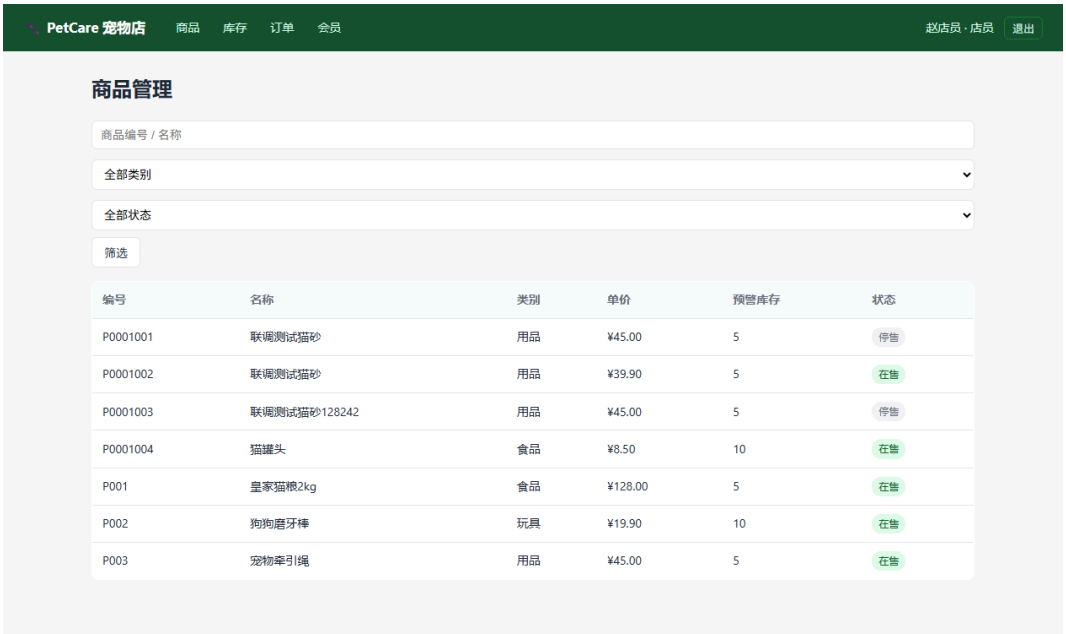
订单详情（金额、明细、重复合并）



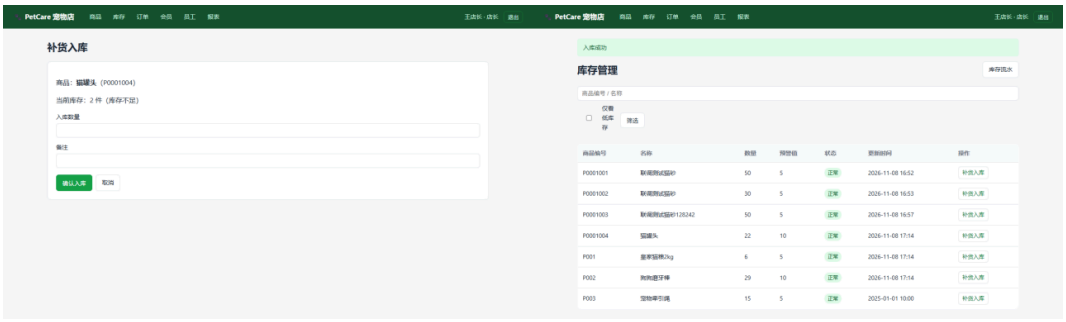
库存与流水（下单后扣减）

3.4 其余模块

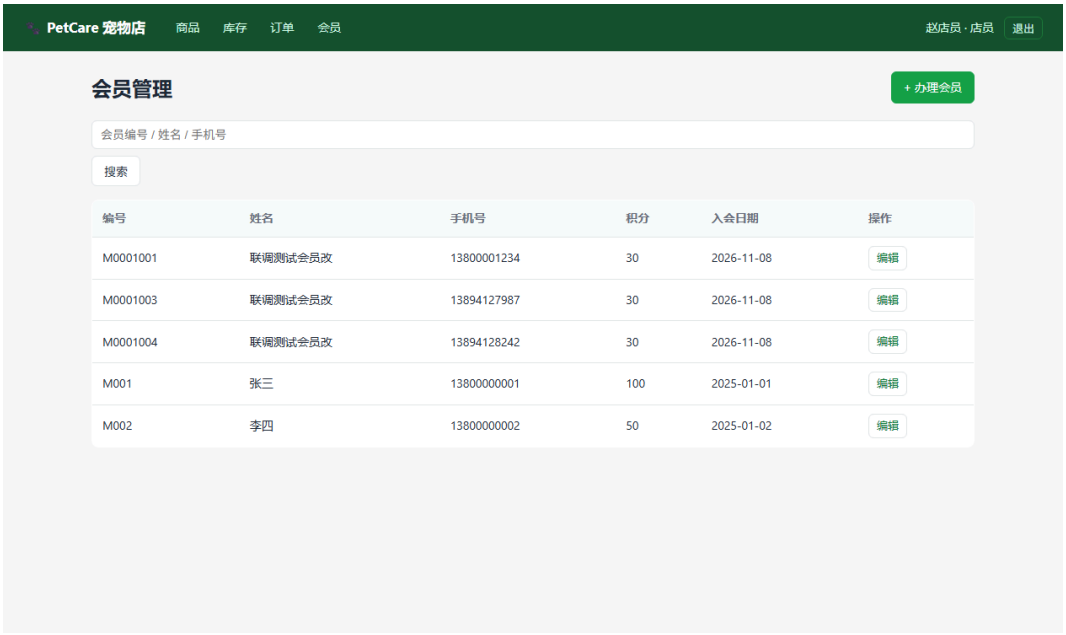
模块	说明
商品	关键字筛选 + 分页；店长可新增（自动建立初始库存）/ 编辑 / 停售
库存	库存状态（正常 / 不足 / 缺货）、流水；店长可补货入库
会员	建档、改资料、积分调整
报表	商品销售、库存状态、会员消费（店长）



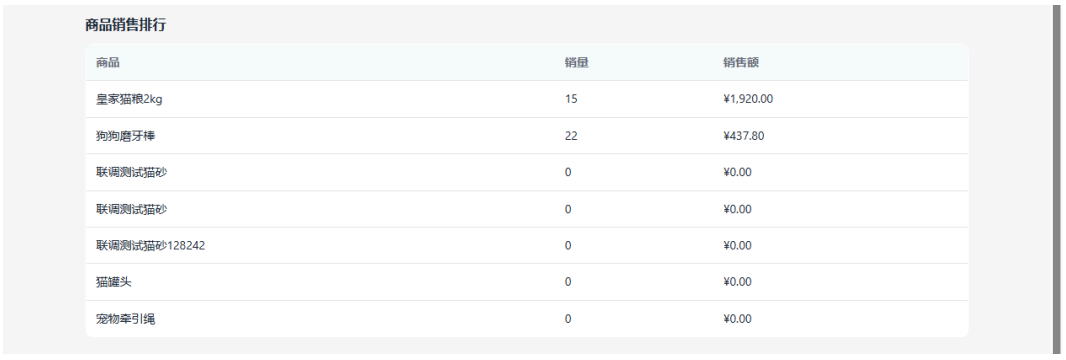
商品列表



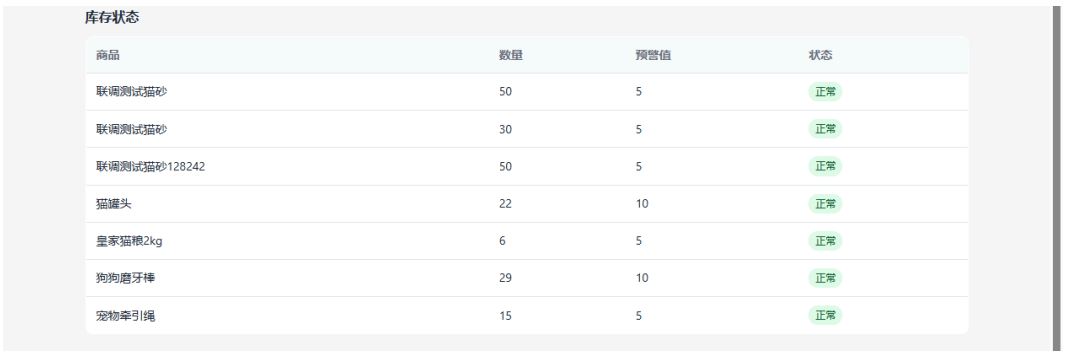
补货入库前后对比



会员管理



报表：商品销售



报表：库存状态



报表：会员消费

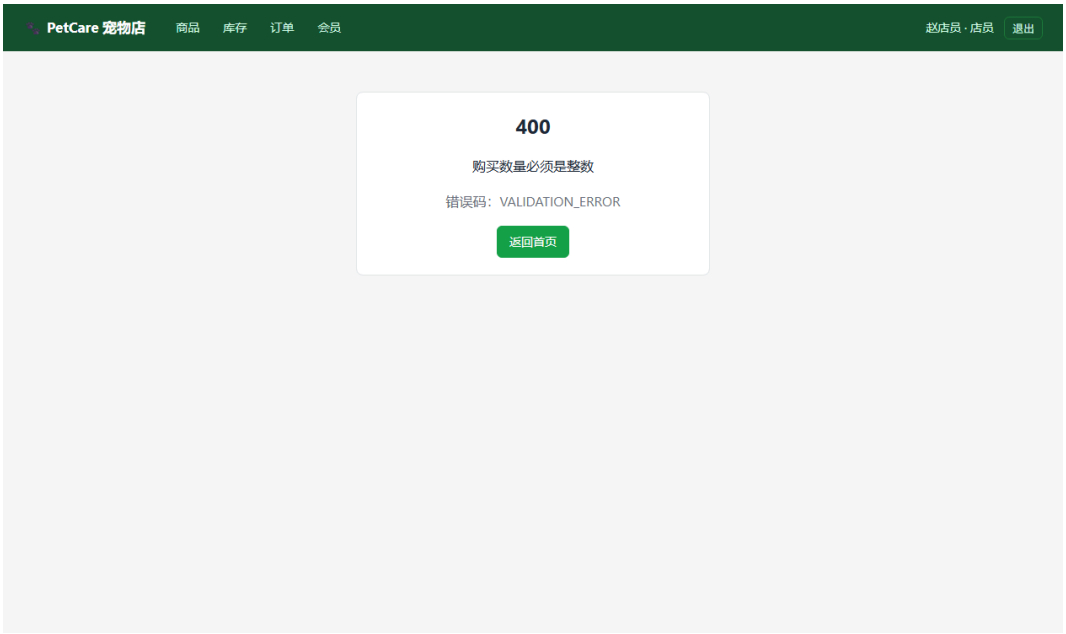
四、安全与验证

4.1 双重防线

防线	实现	位置
应用层	`roles_required("店长")` 拦截非店长的写操作	`app/security.py`
数据库层	受限账号最小权限（如店员无建商品、读员工表权限）	`09_app_accounts.sql`

4.2 非法输入校验

业务层以 `parse_int` / `parse_decimal` 统一校验表单输入，非法输入返回 `VALIDATION_ERROR`（HTTP 400）并提示友好文案，不展示任何 SQL、堆栈或内部信息。



数量非整数 → 400 提示

4.3 越权访问拦截

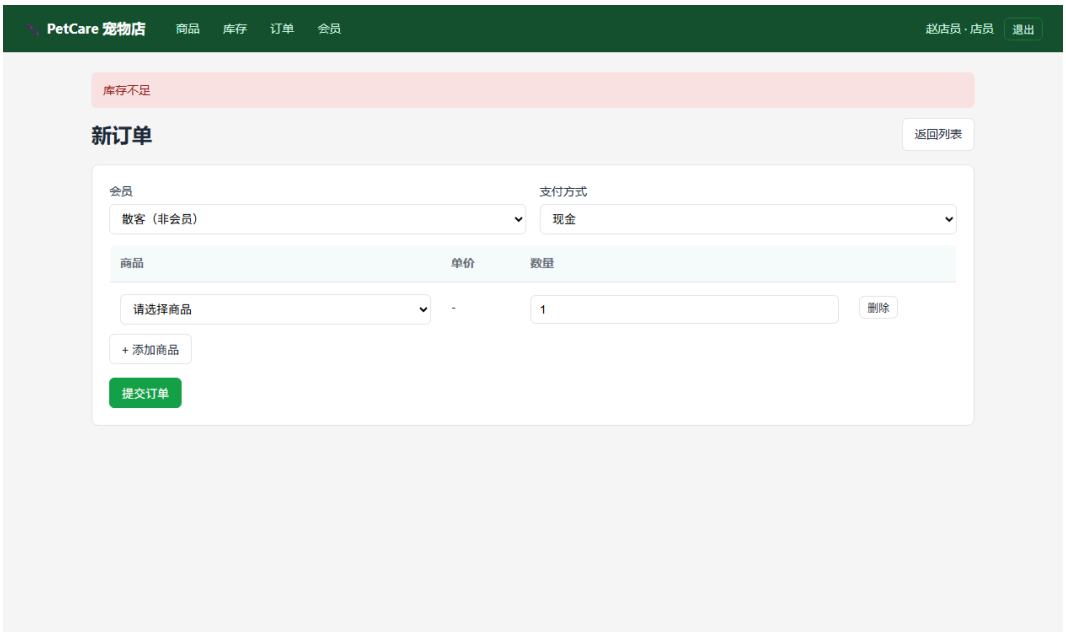
店员直接访问店长专属路由（如 `/employees`、`/products/new`）时被拦截，提示「没有执行该操作的权限」并跳回工作台。



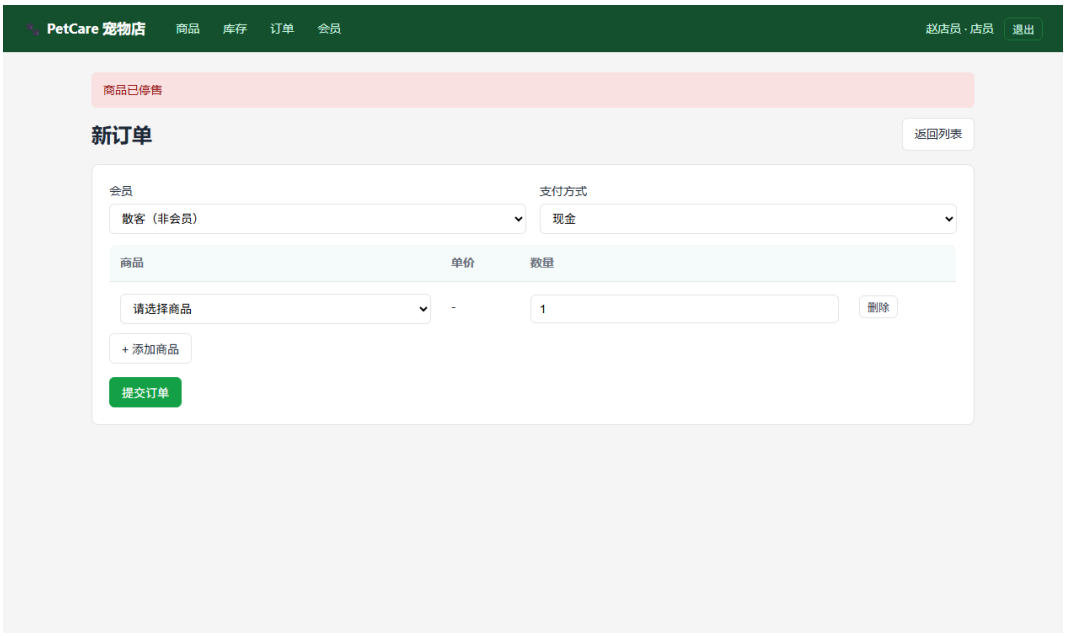
店员越权访问被拦截

4.4 统一错误反馈

- 页面以 `flash` 展示友好提示；
- `Accept: application/json` 的请求返回统一 JSON 结构：`{"ok": false, "code": "...", "message": "...", "data": null, "request_id": "..."}`；
- 错误码到 HTTP 状态码的映射统一维护（如 `VALIDATION_ERROR`→400、`FORBIDDEN`→403、`BUSINESS_RULE_VIOLATION`→409）；
- 后端日志记录 `request_id / operation / role / code / message / detail`。



库存不足 → 整单回滚提示



停售商品下单被拒

```
# PetCare 后端日志样例（logs/app.log，节选真实运行记录）
# 行格式：<时间> <级别> <logger> <message>
#
# 访问结果行 message 字段：  request_id / method / path / endpoint / role / status
# 错误行     message 字段：  request_id / operation / role / code / message / detail
# role 取值：店员 / 店长 / -（未登录）
#
# ---- 业务错误：参数非法（表单数量非整数）----
2026-11-08 17:14:04,832 ERROR petcare.web error request_id=e9b2261982c3 operation=orders.create role=店员 code=VALIDATION_ERROR message='购买数量必须是整数'
detail=None
2026-11-08 17:14:04,833 INFO  petcare.web request_id=e9b2261982c3 method=POST path=/orders/new endpoint=orders.create role=店员 status=400
#
# ---- 业务错误：库存不足（整单回滚）----
2026-11-08 17:14:05,970 ERROR petcare.web error request_id=b050bb4747db operation=orders.create role=店员 code=BUSINESS_RULE_VIOLATION message='库存不足'
detail=None
2026-11-08 17:14:05,972 INFO  petcare.web request_id=b050bb4747db method=POST path=/orders/new endpoint=orders.create role=店员 status=400
#
# ---- 业务错误：停售商品下单被拒 ----
2026-11-08 17:14:07,191 ERROR petcare.web error request_id=020441b1375d operation=orders.create role=店员 code=BUSINESS_RULE_VIOLATION message='商品已停售'
detail=None
2026-11-08 17:14:07,193 INFO  petcare.web request_id=020441b1375d method=POST path=/orders/new endpoint=orders.create role=店员 status=400
#
# ---- 权限拦截：店员访问店长专属 /employees -> 302 重定向 ----
2026-11-08 17:14:03,531 INFO  petcare.web request_id=3f646e613f05 method=GET path=/employees endpoint=employees.list role=店员 status=302
#
# ---- 登录与正常访问 ----
2026-11-08 17:14:12,603 INFO  petcare.web request_id=4473737f630f method=POST path=/login endpoint=auth.login role=店长 status=302
2026-11-08 17:14:15,381 INFO  petcare.web request_id=b3e122eee543 method=GET path=/reports endpoint=reports.index role=店长 status=200
```

后端结构化日志

4.5 测试与验证结果

验证项	结果
应用侧集成测试（mock DAO）	41 项通过
真实 SQL Server + Flask 全流程联调	29 步全部通过
数据侧集成测试	63 项通过
界面与异常截图	18 张

五、总结

已完成：分层 Web 应用（页面层 / 业务层 / 统一异常与日志）、登录鉴权、商品/库存/订单/会员/员工/报表、店员与店长的角色界面与权限差异，以及非法输入 / 越权 / 错误反馈的验证；业务层严格按 DAO 契约实现、不编写 SQL。

遗留问题（v1.1）：服务预约尚未进入应用；订单「至少一条明细」「同订单同商品唯一」目前由 DAO 保证、数据库未强制；`Orders.total_amount` 为受控冗余、由事务与回归保证一致。

附录：数据恢复（演示结束后执行）

现场演示会写入新订单、扣减库存。恢复方式为「重建主库」：`sql/01_create_database.sql` 是 `DROP + CREATE`，重跑建库脚本即回到干净基线，无需提前备份。

```
python scripts/run_sql.py sql/01_create_database.sql --database master --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/02_create_tables.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/03_insert_data.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/06_view.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/07_constraint.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/08_role.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/migration.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/09_app_accounts.sql --driver "ODBC Driver 17 for SQL Server"
python scripts/run_sql.py sql/regression.sql --driver "ODBC Driver 17 for SQL Server"
```

其中 `--driver` 需改为本机实际安装的驱动（`python -c "import pyodbc; print(pyodbc.drivers())"` 可查），`.env` 的 `DB_DRIVER` 与之保持一致。

- `08_role.sql` 会故意报 229 权限错误并停在 ERROR，属预期，继续往下跑即可。
- `09_app_accounts.sql` 幂等（`IF NOT EXISTS`），重复跑不会报「登录已存在」。
- 最后 `regression.sql` 输出 `REGRESSION PASS` 即恢复完成。